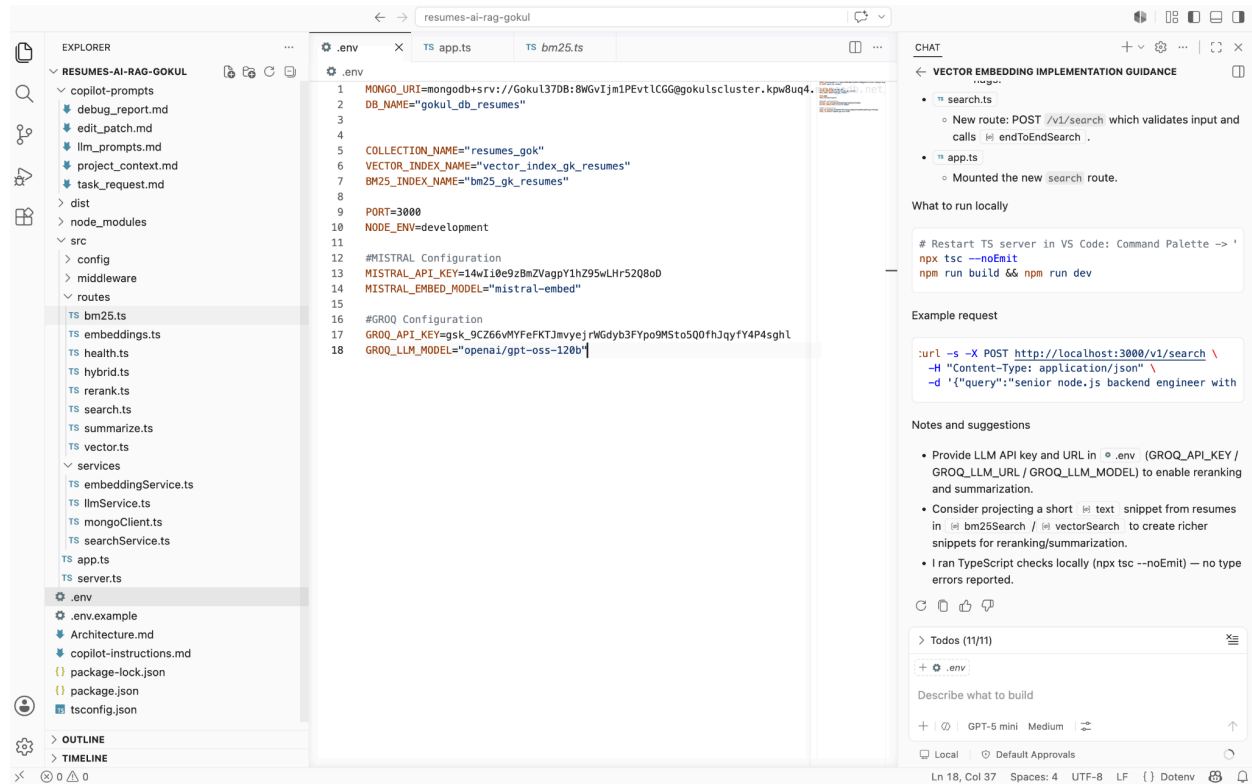


Complete the Resume RAG, backend implementation and to Validate it

Note: The Langflow ingestion of resumes are **NOT** done in this assignment, this involves only retrieval of data via backend and validating the backend implementation.

The **RAG** project created via vibe coding:



.env file details:

```
MONGO_URI=mongodb+srv://Goku137DB:8W6vIjm1PEvtLCGG@gokulscluster.kpw8uq4.mongodb.net/?appName=Goku137DB
DB_NAME="goku1_db_resumes"

COLLECTION_NAME="resumes_gok"
VECTOR_INDEX_NAME="vector_index_gk_resumes"
BM25_INDEX_NAME="bm25_gk_resumes"

PORT=3000
NODE_ENV=development

#MISTRAL Configuration
MISTRAL_API_KEY=14wIi0e9zBmZVagpY1hZ95wLHr52Q8oD
MISTRAL_EMBED_MODEL="mistral-embed"

#GROQ Configuration
GROQ_API_KEY=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
GROQ_LLM_MODEL="openai/gpt-oss-120b"
```

Few commands run : npm tsc --noEmit, npm run build & npm run dev:

EXPLORER

- RESUMES-AI-RAG-GOKUL
 - copilot-prompts
 - debug_report.md
 - edit_patch.md
 - llm_prompts.md
 - project_context.md
 - task_request.md
 - dist
 - node_modules
 - src
 - config
 - middleware
 - routes
 - bm25.ts
 - embeddings.ts
 - health.ts
 - hybrid.ts
 - rerank.ts
 - search.ts
 - summarize.ts
 - vector.ts
 - services
 - embeddingService.ts
 - llmService.ts
 - mongoClient.ts
 - searchService.ts
 - app.ts
 - server.ts
 - .env
 - .env.example
 - Architecture.md
 - copilot-instructions.md
 - package-lock.json
 - package.json
 - tsconfig.json

Source Control

.env

```
1 MONGO_URI=mongodb+srv://Goku137DB:8WgVj1m1PEvtLCG@goku1sclo
2 DB_NAME="goku1_db_resumes"
3
4
5 COLLECTION_NAME="resumes_gok"
6 VECTOR_INDEX_NAME="vector_index_gk_resumes"
7 BM25_INDEX_NAME="bm25_gk_resumes"
8
9 PORT=3000
10 NODE_ENV=development
11
12 #MISTRAL Configuration
13 MISTRAL_API_KEY=14wI10e9z8mZVagpY1hZ95wLHr52Q8oD
14 MISTRAL_EMBED_MODEL="mistral-embed"
15
16 #GROQ Configuration
17 GROQ_API_KEY=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
18 GROQ_LLM_MODEL="openai/gpt-oss-120b"
```

Terminal

```
karthikradhakrishnan@Karthiks-MacBook-Air resumes-ai-rag-gok
ul %
karthikradhakrishnan@Karthiks-MacBook-Air resumes-ai-rag-gok
ul % npx tsc --noEmit
karthikradhakrishnan@Karthiks-MacBook-Air resumes-ai-rag-gok
ul % npm run build
> resumes-ai-rag@0.1.0 build
> tsc
karthikradhakrishnan@Karthiks-MacBook-Air resumes-ai-rag-gok
ul % npm run dev
> resumes-ai-rag@0.1.0 dev
> ts-node-dev --respawn --transpile-only src/server.ts
[INFO] 23:54:29 ts-node-dev ver. 2.0.0 (using ts-node ver. 1
0.9.2, typescript ver. 5.9.3)
MongoDB connected
Server listening on port 3000
```

CHAT

VECTOR EMBEDDING IMPLEMENTATION GUIDANCE

- Calls rerankCandidates (LLM) on top N.
- Optionally summarizes top candidates with summarizeCandidateFit .
- Returns results, summaries, timings, and fallback flags.

- search.ts
 - New route: POST /v1/search which validates input and calls endToEndSearch .
- app.ts
 - Mounted the new search route.

What to run locally

```
# Restart TS server in VS Code: Command Palette -> "TypeScript: Res
npx tsc --noEmit
npm run build && npm run dev
```

Example request

```
curl -s -X POST http://localhost:3000/v1/search \
-H "Content-Type: application/json" \
-d '{"query":"senior node.js backend engineer with mongo","topK":
```

Notes and suggestions

- Provide LLM API key and URL in .env (GROQ_API_KEY / GROQ_LLM_URL / GROQ_LLM_MODEL) to enable reranking and summarization.
- Consider projecting a short text snippet from resumes in bm25Search / vectorSearch to create richer snippets for reranking/summarization.
- I ran TypeScript checks locally (npx tsc --noEmit) — no type errors reported.

Todos (11/11)

- .env

Describe what to build

+ Agent GPT-5 mini Medium

Health check:

COLLECTIONS

- GenAI-March
- GenAI-March-Home Work
- Grocery Store Handson API
- JiraCollection_Fetch_User_Story
- JiraCollection_Langflow_Week6
- Langflow API
- Loyalty Enrol User
- RAG MongoDB - Data Ingestion
- Resumes_AI_Rag
 - embedding
 - POST mistral_embedding
 - search
 - POST BM25_search
 - POST vector_search
 - POST hybrid_search
 - POST end_2_end
 - rerank
 - POST llm_rerank_minimal
 - POST llm_rerank_full_content
 - summarize
 - POST summarize
 - health
 - GET health
 - GET DB_health

REST Client

GET http://localhost:3000/v1/health

Body

```
{
  "name": "resumes-ai-rag",
  "version": "0.1.0",
  "uptimeSeconds": 98
}
```

DB Health Check:

COLLECTIONS

GenAI-March

GenAI-March-Home Work

Grocery Store Handson API

JiraCollection_Fetch_User_Story

JiraCollection_Langflow_Week6

Langflow API

Loyalty Enrol User

RAG MongoDB - Data Ingestion

Resumes_AI_Rag

embedding

mistral_embedding

search

BM25_search

vector_search

hybrid_search

end_2_end

rerank

llm_rerank_minimal

llm_rerank_full_content

summarize

summarize

health

health

DB_health

Trello Api Study

Trello Other Study

Trello Test Assignment 1

Trello Test Assignment 2

Valentines Book Handson Assignment

ENVIRONMENTS

SPECS

FLOWs

Resumes_AI_Rag > health > DB_health

GET http://localhost:3000/v1/health/db

Send

Docs Params Authorization Headers 6 Body Scripts Settings

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 7 Test Results

200 OK 38 ms 281 B Save Response

JSON

Preview Visualize

```
{
  "status": "ok",
  "ping": {
    "ok": 1
  },
  "latencyMs": 33
}
```

Mistral Embedding:

COLLECTIONS

GenAI-March

GenAI-March-Home Work

Grocery Store Handson API

JiraCollection_Fetch_User_Story

JiraCollection_Langflow_Week6

Langflow API

Loyalty Enrol User

RAG MongoDB - Data Ingestion

Resumes_AI_Rag

embedding

mistral_embedding

search

BM25_search

vector_search

hybrid_search

end_2_end

rerank

llm_rerank_minimal

llm_rerank_full_content

summarize

summarize

health

health

DB_health

Trello Api Study

Trello Other Study

Trello Test Assignment 1

Trello Test Assignment 2

Valentines Book Handson Assignment

ENVIRONMENTS

SPECS

FLOWs

POST http://localhost:3000/v1/embeddings

Send

Docs Params Authorization Headers 8 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

Schema Beautify

```
{
  "input": "hello world"
}
```

Body Cookies Headers 7 Test Results

200 OK 378 ms 19.2 KB Save Response

JSON

Preview Visualize

```
1012 -0.07080078125,
1013 -0.004718780517578125,
1014 0.0550537189375,
1015 -0.005931854248046875,
1016 0.02569580078125,
1017 -0.02923583984375,
1018 -0.042205810546875,
1019 0.0201873779296875,
1020 0.0211029052734375,
1021 -0.00095221366882324219,
1022 0.004261016845703125,
1023 0.04876708984375,
1024 -0.010223380671875,
1025 0.00832360943359375,
1026 -0.0001577138900756836,
1027 -0.0121917724609375,
1028
1029 ]
```

Response Time

377.92 ms

Prepare

4.43 ms

Socket Initialization

0.34 ms

DNS Lookup

0.10 ms

TCP Handshake

0 ms

Waiting (TTFB)

375.91 ms

Download

1.47 ms

Process

0.19 ms

Vector search, response empty as no ingestion is done:

The screenshot shows a REST client interface with a sidebar on the left listing various collections and endpoints. The main panel displays a POST request to `http://localhost:3000/v1/search/vector`. The request body is a JSON object with the following structure:

```
1 {
2   "query": "Give me selnum prfile with experince in banking domin",
3   "topK": 10,
4   "filters": {
5     "minYearsExperience": 10
6   }
7 }
8
```

The response is a 200 OK status with a response time of 852 ms and a body size of 323 B. The response body is a JSON object with the following structure:

```
1 {
2   "query": "Give me selnum prfile with experince in banking domin",
3   "topK": 10,
4   "results": []
5 }
```

BM25 search, response empty as no ingestion is done:

The screenshot shows a REST client interface with a sidebar on the left listing various collections and endpoints. The main panel displays a POST request to `http://localhost:3000/v1/search/bm25`. The request body is a JSON object with the following structure:

```
1 {
2   "query": "Senior Node.js Developer with expertise in MongoDB",
3   "topK": 10,
4   "filters": {
5     "minYearsExperience": 3
6   }
7 }
8
```

The response is a 200 OK status with a response time of 39 ms and a body size of 320 B. The response body is a JSON object with the following structure:

```
1 {
2   "query": "Senior Node.js Developer with expertise in MongoDB",
3   "topK": 10,
4   "results": []
5 }
```

Hybrid search, response empty as no ingestion is done:

The screenshot shows a REST client interface with a sidebar on the left listing collections. The main panel displays a POST request to `http://localhost:3000/v1/search/hybrid`. The request body is a JSON object:

```
1 {
2   "query": "senior node.js backend engineer",
3   "topK": 20,
4   "filters": {
5     "minYearsExperience": 3
6   }
7 }
```

The response is a 200 OK status with a response time of 421 ms and a body size of 322 B. The response body is an empty JSON object:

```
1 {}
2 }
```

E2E search, response empty as no ingestion is done:

The screenshot shows a REST client interface with a sidebar on the left listing collections. The main panel displays a POST request to `http://localhost:3000/v1/search/end_2_end`. The request body is a JSON object:

```
1 {
2   "query": "senior node.js backend engineer with MongoDB experience",
3   "topK": 10,
4   "rerankTopK": 8,
5   "filters": { "minYearsExperience": 4 },
6   "summarize": true,
7   "summarizeTopK": 3,
8   "summarizeStyle": "short",
9   "summarizeMaxTokens": 150
10 }
```

The response is a 200 OK status with a response time of 738 ms and a body size of 475 B. The response body is an empty JSON object:

```
1 {}
2 }
```

Rerank, response empty as no ingestion is done:

The screenshot shows a REST client interface with a sidebar on the left listing collections. The main panel displays a POST request to `http://localhost:3000/v1/search/rerank`. The request body is a JSON object with a query and candidates. The response is empty, indicating no ingestion.

```
1 {
2   "query": "senior node.js backend engineer with MongoDB experience",
3   "candidates": [
4     {
5       "resumeId": "691db80aa895776f97b6eca6",
6       "snippet": "Experienced Node.js backend developer with 6 years building REST APIs and integrating MongoDB...",
7       "score": 2.3
8     },
9     {
10      "resumeId": "691db80aa895776f97b6eca7",
11      "snippet": "Fullstack engineer (Node.js, Express) with production MongoDB and replication experience...",
12      "score": 1.9
13    }
14  ],
15  "topK": 10
16 }
```

The response is empty, indicating no ingestion.

Summarize, response empty as no ingestion is done:

The screenshot shows a REST client interface with a sidebar on the left listing collections. The main panel displays a POST request to `http://localhost:3000/v1/search/summarize`. The request body is a JSON object with a query and candidate. The response is empty, indicating no ingestion.

```
1 {
2   "query": "Senior Node.js backend engineer with MongoDB experience",
3   "candidate": {
4     "resumeId": "691db80aa895776f97b6eca6",
5     "name": "Ashwin P",
6     "snippet": "Built scalable Node.js REST APIs, optimized MongoDB queries, designed sharding and indexing strategies..."
7   },
8   "style": "detailed",
9   "maxTokens": 150
10 }
```

The response is empty, indicating no ingestion.